

VIP Preferred — Automation Framework KT Guide

Product: VIP Connect (VIP Preferred SDK)
Audience: QA Engineers onboarding to SDK automation
Author: Brock Pivirotto
Last Updated: July 2026

Overview

Two separate C# MSTest automation frameworks cover the VIP Preferred product:

Framework	Repo	Focus	Test Runner
API Framework	<code>vip-preferred-api/SDKAutomationFramework</code>	RestSharp API tests against backend endpoints	MSTest
UI Framework	<code>vip-preferred-sdk/SDKAutomatedTests/SDK_Automation_Framework</code>	Selenium end-to-end UI tests via Mock Operator	MSTest

Both frameworks use the same design principles: environment-driven configuration, named test patrons (SessionTypes), and a shared helper model for session creation.

Part 1: API Framework

1.1 Tech Stack

Tool	Purpose
C# / .NET 8	Language and runtime

Tool	Purpose
MSTest	Test runner ([TestClass], [TestMethod])
RestSharp	HTTP client for API calls
Newtonsoft.Json	JSON serialization / deserialization
NLog	Structured test logging
Microsoft.Extensions.Configuration	Reads test-keys.json for tokens and URLs

1.2 Project Structure

SDKAutomationFramework/	
├─ TestApiKeys.cs	─ Environment-specific tokens and base URLs
├─ Tests/	
│ └─ BaseTest.cs	─ Setup/teardown; reads SDK_ENV env variable
│ └─ API/	
│ │ └─ APITests.cs	─ Miscellaneous API tests (add/delete bank)
│ │ └─ SessionID/SessionIDAPITests.cs	─ Session creation + account lookup tests
│ │ └─ Enrollment/EnrollmentAPITests.cs	─ Enrollment flow tests
│ │ └─ Deposit/DepositAPITests.cs	─ Deposit and deposit approval tests
│ │ └─ Withdraw/WithdrawAPITests.cs	─ Withdrawal creation and approval tests
└─ Helpers/	
│ └─ API/APIFunctions.cs	─ All RestSharp HTTP calls (the API layer)
│ └─ Helpers.cs	─ Random data generators
│ └─ userData.cs	─ User/patron data model
│ └─ generatePatronSessionJson.cs	─ JSON body builders for each SessionType
│ └─ Enums/SessionType.cs	─ Named test patron enum
│ └─ JsonClasses/	
│ │ └─ SessionID.cs	─ PatronSessionResponse + Toggles models
│ │ └─ AccountLookup.cs	─ AccountLookup + BankAccount response
└─ PendingTransactionsContent.cs	─ Pending transaction model

1.3 Environment Configuration

The framework reads the `SDK_ENV` environment variable at test startup. In local/debug mode this defaults to `DEV`. The CI/CD pipeline injects the correct value for QA, CERT, or PROD runs.

SDK_ENV	Purpose
DEV	Local development — uses dev tokens and dev API URL

SDK_ENV	Purpose
QA	Internal QA environment
CERT	Certification / UAT environment (pre-prod)
PROD	Production smoke tests only

Tokens and URLs are stored in `test-keys.json` (production/CI) or `test-keys.Development.json` (local). These files are **not committed to source control** — obtain them from the team lead or Azure Key Vault.

Four operator tokens are available per environment: OPERATOR1, OPERATOR2, OPERATOR3, OPERATOR4. Each represents a different operator configuration (different toggle settings).

1.4 API Endpoints Under Test

Endpoint	Method	Purpose
<code>/api/patronsession/new</code>	POST	Create session for a new (unenrolled) patron
<code>/api/patronsession/new/simple</code>	POST	Simplified new patron session creation
<code>/api/patronsession/existing</code>	POST	Create session for an enrolled patron
<code>/api/patronsession/existing/simple</code>	POST	Simplified existing patron session
<code>/api/patronsession/true</code>	GET	Retrieve session info and operator toggles
<code>/api/patronsession/accountlookup</code>	POST	Look up patron by VIP card number or SSN/ID
<code>/api/finicity/geturl</code>	GET	Get Finicity bank-linking URL for session
<code>/api/funds/deposit</code>	POST	Submit a deposit request
<code>/api/funds/withdraw</code>	POST	Queue a withdrawal request
<code>/api/operator/DepositApproval</code>	POST	Approve a pending deposit (operator side)
<code>/api/operator/WithdrawApproval</code>	POST	Approve a pending withdrawal (operator side)
<code>/api/operator/DeclineWithdrawal</code>	POST	Decline a pending withdrawal

Endpoint	Method	Purpose
/api/operator/Transaction	POST	Look up transaction by VIP card number
/api/bank	POST	Add a bank account to a session
/api/bank	DELETE	Remove a bank account from a session
/api/syntrove/enroll	POST	Enroll a patron (Syntrove enrollment API)

1.5 Authentication Model

The API uses two distinct authentication header patterns.

Operator bearer token — used on operator-facing endpoints (session creation, account lookup, approvals):

```
Authorization: Bearer <OPERATOR_TOKEN>
```

Patron session — used on patron-facing endpoints (deposit, withdrawal, bank management) after a session ID has been obtained:

```
Session-Id: <sessionId>  
Authorization: PatronSession
```

1.6 Named Test Patrons (SessionType Enum)

The `SessionType` enum maps to pre-seeded test patrons in each environment. `generatePatronSessionJson` builds the correct JSON body for each type.

SessionType	VIP Card (DEV/QA)	Use Case
Dashboard	7210530120	Existing patron on dashboard — standard existing session

SessionType	VIP Card (DEV/QA)	Use Case
Bank	7210504496	Patron with known bank details — used for bank detail validation
BankFlows	7210705556	Patron for add/delete bank flows
ValidWithdrawal	7210742567	Patron eligible for withdrawal approval
InvalidWithdrawal	7210975245	Patron that triggers withdrawal failure
InvalidDeposit1-6	Various	Patrons for deposit edge cases (bad bank, routing, over limit, etc.)
InvalidBank	7210742552	Patron with invalid bank account for deposit
Random	(generated)	Randomly generated new patron — used for enrollment tests
NoPhone	(generated)	New patron missing mobile phone — expects 400
Prod	7770001069	Production smoke patron — always existing user
CertDeposit	7211032683	CERT-specific deposit patron
WithdrawApproval	7210931633	Patron for withdrawal approval flows

1.7 How a Typical API Test Works

A withdrawal test follows this pattern:

1. **Get a session ID** — call `GetSessionIDExistingUser()` with a named patron JSON body and an operator token. Returns a GUID session ID.
2. **Call the target API** — pass the session ID and test data to the relevant function (e.g., `QueueWithdrawalApproval()`).
3. **Assert the HTTP status code** — happy path asserts `HttpStatusCode.OK`; negative cases assert `BadRequest`, `Unauthorized`, or `NotFound`.
4. **Assert response timing** — smoke tests assert response time under 15 seconds (30 seconds for enrollment).

```
var tid = Helpers.RandomIntString(8, 25);
var sessionID = APIFunctions.GetSessionIDExistingUser(
    generatePatronSessionJson.generatePatronJsonContent(
```

```
        SessionType.ValidWithdrawal, null, "1000", "1000", "12", tid),  
        OPERATOR2_TOKEN);  
  
HttpStatusCode status = APIFunctions.QueueWithdrawalApproval(  
    sessionID, "Bank Of America, Na", "061000052", "*2823");  
Assert.IsTrue(status == HttpStatusCode.OK);
```

1.8 Test Categories

Tests are tagged with MSTest categories for selective runs:

Category	Purpose
SmokeTest	Core happy-path tests — run after every deployment
Regression	Full suite — run before releases
Sanity	Production verification — safe for live environment
API	All API tests (applied at the class level)

1.9 Test Class Inventory

SessionIDAPITests

Test Method	Category	What It Tests
GetSessionID	Smoke/Regression	New patron session returns session ID within 15s
GetSessionIDProd	Sanity	Production new session smoke test
GetFinicityURL	Smoke/Regression	Finicity URL retrieval for existing session
GetSessionIDInvalidTransactionID	Smoke/Regression	Special characters in TID return 400
AccountLookup	Smoke/Regression	VIP card lookup returns 200 within 15s
AccountLookupProd	Sanity	Production account lookup smoke test

Test Method	Category	What It Tests
AccountLookupBankDetails	Smoke/Regression	Full response validated: name, DOB, SSN, address, idType, limits, bank details
AccountLookupNoToken	Regression	No bearer token returns 401
AccountLookupNoBody	Regression	Empty VIP number returns 400
AccountLookupNoAccount	Regression	Unknown VIP number returns 404
GetSessionIDToggles	Regression	Session response contains all operator toggle flags
GetSessionIDNoPhone	Regression	Missing mobile phone throws exception
GetSessionIDNoBody	Regression	Empty JSON body throws exception
GetSessionIDNoBearerToken	Regression	Missing token throws exception
Get_NewSessionSimple	Smoke/Regression	Simple new patron endpoint returns 200 with sessionId
Get_NewSessionSimpleMissingFields	Smoke/Regression	Missing email returns 400 "Validation failed"
Get_ExistingSessionSimple	Smoke/Regression	Simple existing patron returns 200 with sessionId
Get_ExistingSessionSimpleMissingDob	Smoke/Regression	Missing DOB returns 400 with specific validation message
Get_ExistingSessionSimpleMissingVipNumber	Smoke/Regression	Missing VIP number returns 400 with specific message
Get_ExistingSessionSimpleMissingPatronid	Smoke/Regression	Missing PatronId returns 400 with specific message

EnrollmentAPITests

Test Method	Category	What It Tests
AccountLookupBySSN	Smoke/Regression	Lookup by IdType + IdNumber returns 200 with vipCardNumber within 15s
AccountLookupBySSN_NoToken	Regression	Returns 401
AccountLookupBySSN_NoIdNumber	Regression	Returns 400
AccountLookupBySSN_NoAccount	Regression	Returns 404 with errorCode "12" and "Account not found"
Enroll_Success	Smoke/Regression	Full enrollment: 200, status="success", responseCode=0, sessionExpires present
Enroll_NoSessionId	Regression	Empty session header returns 401 or 404
Enroll_InvalidSessionId	Regression	Zero GUID session returns 401 or 404
Enroll_MissingAccountNumber	Regression	Returns 400
Enroll_MissingRoutingNumber	Regression	Returns 400
Enroll_InvalidAccountLimit	Regression	Negative account limit returns 400

DepositAPITests

Test Method	Category	What It Tests
DepositAPI	Smoke/Regression	Successful deposit returns 200 within 15s
DepositBadSessionID	Regression	Empty session ID returns 401
DepositRandomSessionID	Regression	Random GUID session returns 401
DepositInvalidBankName	Regression	Empty bank name returns 400
DepositInvalidRoutingNumber	Regression	Invalid routing number returns 400
DepositInvalidAccountNumber	Regression	Invalid masked account number returns 400

Test Method	Category	What It Tests
<code>DepositInvalidMaxAmount</code>	Regression	Amount over limit returns 400
<code>DepositInvalidMinAmount</code>	Regression	Zero amount returns 400
<code>DepositInvalidNegativeAmount</code>	Regression	Negative amount returns 400
<code>ApproveDeposit</code>	Smoke/Regression	Deposit + operator approval returns 200
<code>ApproveDepositNullRemainingDailyBalance</code>	Smoke/Regression	Approval without RemainingDailyBalance returns 200
<code>ApproveDepositSimple</code>	Smoke/Regression	Simple session endpoint + approval returns 200
<code>ApproveDepositNoTid</code>	Regression	Approval without transaction ID returns 400
<code>ApproveDepositNoToken</code>	Regression	Approval without token returns 401

WithdrawAPITests

Test Method	Category	What It Tests
<code>WithdrawApproval</code>	Smoke/Regression	Queue + approve withdrawal both return 200
<code>WithdrawApprovalNullWalletBalance</code>	Smoke/Regression	Approval with null wallet balance returns 200
<code>WithdrawApprovalSimple</code>	Smoke/Regression	Simple session endpoint + approval returns 200
<code>WithdrawApprovalNoToken</code>	Regression	Approve without token returns 401
<code>WithdrawApprovalNoTid</code>	Smoke/Regression	Approve without TID returns 400
<code>CreateWithdrawal</code>	Smoke/Regression	Queue withdrawal returns 200 within 15s
<code>TransactionLookup</code>	Smoke/Regression	Transaction lookup returns 200 with <code>warrantyMode</code> field present

Test Method	Category	What It Tests
CreateWithdrawalFailure	Smoke/Regression	Bad masked account number returns 400
CreateWithdrawalNoSession	Regression	Empty session returns 401
CreateWithdrawalRandomSession	Regression	Invalid session string returns 401

1.10 Operator Toggle Validation

`GetSessionInfo()` in `APIFunctions` validates that a session response contains all expected operator toggle flags. If any toggle is missing the method returns false. Currently validated toggles:

- `plaidEnabled` — whether Plaid bank linking is available
- `showDashboardButtonAfterSuccess` — dashboard button on success screen
- `manageFundingSourcesInDropdown` — inline bank management
- `showTransactionConfirmation` — confirmation screen toggle
- `identificationInfoEnabled` — ID collection step toggle
- `plaidDefault` — Plaid as the default bank link method
- `mitekEnabled` — Mitek ID verification toggle
- `includeBankInfoOnAccountLookUp` — bank details in account lookup
- `isAmountEditable` — patron can edit transaction amount
- `unmaskBankAccountNumber` — full account number display
- `threatMetrixEnabled` — ThreatMetrix fraud detection toggle
- `threatMetrixContinueOnFail` — fail-open behavior for ThreatMetrix
- `depositApproval` — operator-side deposit approval workflow

Part 2: UI Framework

2.1 Tech Stack

Tool	Purpose
C# / .NET 8	Language and runtime
MSTest	Test runner

Tool	Purpose
Selenium WebDriver	Browser automation (Chrome primary, Firefox available)
RestSharp	API calls for test setup (session creation)
NLog	Test logging
Page Object Model	UI abstraction pattern
WebDriverWait	120-second explicit wait for all element interactions

2.2 Project Structure

SDKAutomationFramework/	
├─ TestApiKeys.cs	– Tokens, URLs, including
HYBRID_OPERATOR_TOKEN	
├─ Tests/	
│ └─ BaseTest.cs	– Env config, WebDriver setup, SDK loader
methods	
├─ Dashboard/	
│ └─ BaseDashboardTest.cs	– Shared setup for dashboard test classes
│ └─ DashboardTests.cs	– Dashboard navigation, help, profile,
session tests	
├─ Deposit/DepositTests.cs	– Deposit flow tests including Hybrid
├─ Withdraw/WithdrawTests.cs	– Withdrawal flow tests
└─ ManageAccounts/ManageAccountTests.cs	– Bank account management tests
└─ Enrollment/EnrollmentTests.cs	– New patron enrollment tests
├─ Pages/	
│ └─ BasePage.cs	– Common Selenium actions
│ └─ MockOperator/MockOperatorPage.cs	– Mock Operator interactions + callback
verification	
├─ Enrollment/	
│ └─ IdentificationInfoPage.cs	– ID information form
│ └─ AddBankAccountPage.cs	– Bank entry (manual, Plaid, Finicity)
│ └─ EnrollmentResponsePage.cs	– Success/failure result page
│ └─ PlaidPage.cs	– Plaid OAuth bank linking
│ └─ FinicityPage.cs	– Finicity bank linking
└─ Dashboard/	
│ └─ DashboardHomePage.cs	– Main dashboard (available funds,
navigation)	
├─ DepositFundsPage.cs	– Deposit transaction page
├─ WithdrawFundsPage.cs	– Withdrawal transaction page
├─ MyAccountsPage.cs	– Bank account management page
├─ ProfileInfoPage.cs	– Patron profile display
└─ DeviceVerificationPage.cs	– Device verification / SMS code entry
├─ Helpers/	
│ └─ API/APIFunctions.cs	– Session creation + device ID management
│ └─ WebDriverFactory.cs	– Chrome/Firefox driver creation
│ └─ generatePatronSessionJson.cs	– JSON body builders (expanded session
types)	
└─ Enums/	
│ └─ SessionType.cs	– Named test patron enum (more types than
API framework)	

└─ BrowserType.cs

— Chrome / Firefox enum

2.3 How UI Tests Work — Architecture

UI tests follow a consistent 5-step pattern:

5. **Get a session ID** — call `LoadMockOperatorSID()` which calls the API to get a session ID, then navigates the browser to the Mock Operator page with that session embedded in the URL.
6. **Interact with the Mock Operator** — click the Withdraw or Deposit button on the Mock Operator page. This triggers the SDK to open in a new browser window.
7. **Switch to the SDK window** — `SwitchToLatestWindow()` moves focus to the new tab/window containing the SDK.
8. **Drive the SDK UI** — interact with page objects (enrollment forms, deposit pages, bank dropdowns, etc.).
9. **Verify callbacks** — after a transaction, navigate to the Mock Operator Notifications page to verify the operator received the correct callback payload.

2.4 The Mock Operator Application

The Mock Operator is a separate internal test application that simulates a real operator portal. It has two key pages:

- `/Wc?sessionId=...&TransactionType=...&access_token=...` — Launches the SDK component for a session
- `/Notifications?access_token=...` — Displays the callback notifications received from the API

Tests heavily use `MockOperatorPage` methods to verify callback payloads:

MockOperatorPage Method	What It Verifies
<code>transactionVisible(tid)</code>	Transaction ID appears in notifications table
<code>warrantyModeVisible(tid)</code>	Callback contains WARRANTY=true
<code>nonwarrantyModeVisible(tid)</code>	Callback contains NOWARRANTY=false
<code>cancelMessageDepositCorrect(tid)</code>	Cancel callback: ErrorCode=1000, User Exited Flow, Stage=Deposit
<code>cancelMessageWithdrawCorrect(tid)</code>	Cancel callback: ErrorCode=1000, User Exited Flow, Stage=Withdrawal
<code>cancelMessageEnrollmentCorrect(tid)</code>	Cancel callback: tid present + "User Exited Flow" text
<code>InstantPaymentEligibleVisible(value)</code>	Callback contains InstantPaymentEligible=true/false

MockOperatorPage Method	What It Verifies
<code>patronIDVisible</code>	PatronId field present in callback
<code>returnUrlVisible(tid)</code>	TID present in return URL redirect
<code>transactionTypeVisible(type)</code>	deposit or withdraw type present

2.5 Session Loading Methods in BaseTest

Method	When To Use
<code>LoadMockOperatorSID(sessionType, token, ...)</code>	Load existing patron via Mock Operator
<code>LoadMockOperatorNewSID(jsonContent, token, ...)</code>	Load new patron via Mock Operator
<code>LoadMockOperatorNotifications(token)</code>	Navigate to Mock Operator notifications page
<code>LoadSDK(sid)</code>	Navigate directly to SDK URL (bypasses Mock Operator)
<code>IsApiAvailable(token)</code>	Health check — polls account lookup for 30s before running test

2.6 Operator Tokens in the UI Framework

The UI framework has one additional token compared to the API framework:

- **OPERATOR1** — Standard operator, Plaid enabled, instant payments enabled
- **OPERATOR2** — Finicity enabled, deposit approval workflow enabled
- **OPERATOR3** — Manual bank only (no Plaid/Finicity), editable amounts enabled
- **OPERATOR4** — Plaid prominent toggle enabled
- **HYBRID_OPERATOR_TOKEN** — Hybrid Smart Routing operator (WARRANTY/NOWARRANTY mode)

2.7 Test Class Inventory

EnrollmentTests

Test Method	Category	What It Tests
InvalidInfoOperator	Regression	Invalid patron data from operator shows enrollment incomplete screen
EnrollmentPlaid	Smoke/Regression	New patron → Plaid → enrolled → withdraw page loads
EnrollmentFinicity	Smoke/Regression	New patron → Finicity → enrolled → withdraw page loads
ProductRoute	(none)	VIPP bank added → VIPP logo shown; VIPO bank added → logo switches to VIPO
VIPOEnrollmentWithID	[Ignore]	VIPO enrollment with ID document flow
EnrollmentExitFlow	Regression	Cancel before bank added → "User Exited Flow" enrollment callback
EnrollmentExitFlowAddedBank	Regression	Cancel after bank added → "User Exited Flow" enrollment callback
EnrollmentPlaidProminentToggle	Smoke/Regression	Plaid prominent toggle: Plaid shown non-default, manual entry visible
EnrollmentManual	Smoke/Regression	Manual bank entry → enrolled → withdraw page loads
EnrollmentBankMode	Smoke/Regression	ManageBanks transaction type → enrolled → My Accounts loads
TermsAndConditions	(none)	Terms and conditions element visible after bank entry
PlaidDisabledTest	Regression	Operator with Plaid disabled — Plaid button not visible
FrontendFieldValidation	(none)	Field presence and validation on ID info and bank info pages

DashboardTests

Test Method	Category	What It Tests
dashboardHelpButton	Regression	Help, profile, info buttons functional; logout works on deposit + withdraw
dashboardProfileIcon	Regression	Patron name shown on all pages; full deposit and withdraw flows verified
dashboardTermsandConditionsFooter	Regression	Terms footer visible on dashboard, deposit, withdraw, manage accounts
AvailableFunds	Regression	Available funds displayed; comma formatting; logout sends cancel callback
Dashboard	Sanity	Production sanity: existing patron loads deposit page
InvalidAccountInquiry	(none)	Invalid patron shows E-99 error and sends callback
InvalidateSession	Regression	Re-using same patron in second tab invalidates first (session expired shown)
DashboardErrorMessageing	[Ignore]	E-16, E-17, E-19, E-26, E-92, E-99 error codes via JS injection

DepositTests

Test Method	Category	What It Tests
Deposit	Smoke/Regression	Limit shown, bank dropdown, deposit completes, available funds updates, callback
DepositWarrantyModeOverride	Regression	Hybrid: WARRANTY mode deposit — callback shows WARRANTY=true
DepositWarrantyModeOverrideNoWarranty	Regression	Hybrid: NOWARRANTY mode deposit — callback shows NOWARRANTY=false
DepositDeviceVerification	(none)	SMS verification code flow via TextGrid — code entered, deposit page loads

Test Method	Category	What It Tests
<code>returnurlQueryStrignParam</code>	Regression	Return URL query string params appended after deposit
<code>DepositApproval</code>	Regression	Approval workflow: deposit shows "in progress" state, callback received
<code>DepositEditAmount</code>	Smoke/Regression	Manual amount entry, quick amount buttons, deposit completes
<code>DepositCancelFlow</code>	Smoke/Regression	Cancel: No stays on page; Yes sends "User Exited Flow" deposit callback
<code>DepositDuplicateTransaction</code>	Regression	Duplicate TID on same patron shows E-92 nested error
<code>DepositOneshot</code>	Regression	Auto-redirect to operator after 5s with TID and "deposit" type in URL
<code>TransactionNoRedirect</code>	Regression	Back to Operator button redirects with TID and "deposit" type

WithdrawTests

Test Method	Category	What It Tests
<code>WithdrawFunds</code>	Smoke/Regression	Full withdraw: completes, callback shows InstantPaymentEligible=false and PatronId
<code>WithdrawFundsEditAmount</code>	Smoke/Regression	Custom amount, completes, callback shows InstantPaymentEligible=false
<code>WithdrawFundsRTP</code>	(none)	RTP/instant withdraw: completes, callback shows InstantPaymentEligible=true
<code>WithdrawFundsCancelFlow</code>	Regression	Cancel: No stays on page; Yes sends "User Exited Flow" withdrawal callback

Test Method	Category	What It Tests
WithdrawFundsDecimalValue	Regression	Decimal transaction amount (e.g. 13.5) processes successfully
WithdrawFundsOneshot	Regression	Auto-redirect with TID after 5 seconds
DeleteBankFailureActions	[Ignore]	Delete bank failure: try again and complete withdraw fallback flows

ManageAccountTests

Test Method	Category	Notes
AddBankFailureActions	[Ignore]	Add bank failure handling — try again / complete flow
AddDeleteBank	[Ignore]	Full add/delete bank in withdraw and deposit contexts
DefaultAccount	[Ignore]	Set default bank changes available funds displayed
Plaid0BalanceAccountAdd	[Ignore]	Plaid zero-balance account shows error message
TransactionAmountOverDefaultLimit	[Ignore]	Insufficient funds message when transaction exceeds bank balance
BankDropdownActions	Regression	Manage funding source visible; add source visibility based on operator toggle
DeleteLastBank	Smoke/Regression	Cannot delete last bank — minimum limit message shown
AddMoreThanFourBanks	Regression	Shows error when patron has 4 banks (at max limit)
ManageBankMode	Regression	ManageBanks transaction type loads My Accounts page directly

Part 3: Hybrid Smart Routing

3.1 What Hybrid Smart Routing Is

Hybrid Smart Routing dynamically determines whether a transaction is processed through a **warrantied** or **non-warrantied** pathway based on real-time evaluation of the patron's account and bank data. The operator can also override the routing mode explicitly via the session creation request.

3.2 How It Is Passed in the Session

The warranty mode is passed as a parameter in the session creation JSON body via the `warrantyMode` field. Hybrid sessions also use `productType = "1"` (Online Banking / VIPO product).

When using `LoadMockOperatorSID()` in UI tests, the last two parameters are `productType` and `warrantyMode`:

```
LoadMockOperatorSID(account, HYBRID_OPERATOR_TOKEN,  
    "299", "299", fundsToAdd, ltid, "Deposit", "1", "WARRANTY");
```

3.3 Hybrid Test Coverage Summary

Test	Framework	Type	Mode	What Is Asserted
<code>DepositWarrantyModeOverride</code>	UI	Deposit	WARRANTY	Callback contains WARRANTY=true
<code>DepositWarrantyModeOverrideNoWarranty</code>	UI	Deposit	NOWARRANTY	Callback contains NOWARRANTY=false
<code>TransactionLookup</code>	API	Transaction lookup	N/A	Response body contains <code>warrantyMode</code> field
<code>ProductRoute</code>	UI	Enrollment	N/A	VIPP vs VIPO product logo switches after bank type added

3.4 Hybrid-Specific Configuration

The `HYBRID_OPERATOR_TOKEN` is separate from the standard operator tokens and must be configured in `test-keys.json`. It is present in DEV, QA, and CERT environments.

Part 4: Running the Tests

4.1 Prerequisites

10. Clone the repo locally
11. Obtain `test-keys.Development.json` from the team lead or Azure Key Vault and place it in the `SDKAutomationFramework` project root
12. For local runs, `SDK_ENV` defaults to `DEV` in debug mode (set automatically by `#if DEBUG` in `BaseTest`)
13. For UI tests: Chrome must be installed and ChromeDriver version must match the installed Chrome version

4.2 Running via Visual Studio

14. Open the solution in Visual Studio
15. Build the solution
16. Open **Test Explorer** (Test menu → Test Explorer)
17. Filter by Category (SmokeTest, Regression, etc.) or by class name
18. Right-click → **Run Selected Tests**

4.3 Running via CLI (CI/CD)

```
dotnet test --filter "TestCategory=SmokeTest" -- SDK_ENV=CERT
dotnet test --filter "TestCategory=Regression" -- SDK_ENV=QA
dotnet test --filter "TestCategory=Sanity" -- SDK_ENV=PROD
```

4.4 Common Local Failures

Problem	Fix
<code>test-keys.json</code> not found	Copy <code>test-keys.Development.json</code> to the project root
Unable to retrieve session ID	Check that <code>SDK_ENV</code> matches an environment with valid tokens
Chrome driver version mismatch	Download the correct ChromeDriver matching your Chrome version
UI tests time out at Mock Operator	Ensure <code>MOCKOPERATORURL</code> is reachable — VPN may be required
<code>Service Unavailable</code> assertion	API health check failed — the target environment API is down

Part 5: Known Gaps and Notes

5.1 Ignored Tests

The following tests are marked `[Ignore]` and are excluded from all test runs:

- `AddDeleteBankAPI` (API) — add/delete bank via API
- `AddBankFailureActions`, `AddDeleteBank`, `DefaultAccount`, `Plaid0BalanceAccountAdd`, `TransactionAmountOverDefaultLimit` (UI) — bank management flows requiring specific data state
- `DeleteBankFailureActions` (UI) — delete bank failure handling
- `VIPOEnrollmentWithID` (UI) — VIPO-specific enrollment with ID document

These tests may need re-enabling and updating when the corresponding features are under active development.

5.2 Known Bug in Toggle Validation

In `APIFunctions.GetSessionInfo()`, several toggle presence checks contain a copy-paste bug: the condition variable `!toggleh` is reused for every toggle rather than checking each toggle's specific value. The check passes as long as `plaidDefault` (the last toggle whose value is actually stored in `toggleh`) is present in the response. This is a known issue in the codebase.

5.3 Hardcoded Test Data

Several tests contain hardcoded VIP card numbers, PatronIds, DOBs, and bank account details tied to specific patrons seeded in each environment. If these patrons are modified or removed (e.g., limit changes, bank account deletions), the corresponding tests will fail. These are fixed test data records that should not be altered in any environment.

5.4 Response Time Assertions

Smoke tests assert API responses complete within 15 seconds (30 seconds for enrollment). These assertions may produce false failures when the target environment is under load.

Appendix A: SessionType Reference (UI Framework)

The UI framework `generatePatronSessionJson` supports additional session types beyond the API framework. Key types relevant to Hybrid and advanced testing:

SessionType	Notes
<code>warrantyMode</code>	CERT environment patron for Hybrid WARRANTY tests
<code>warrantyModeQA</code>	QA/DEV environment patron for Hybrid WARRANTY tests
<code>DeviceID</code>	CERT patron for device verification tests
<code>DeviceIDQA</code>	QA/DEV patron for device verification tests
<code>ProfileIcon</code>	Patron with name data for profile display tests
<code>RTPwithdraw</code>	Patron with instant payment eligible bank for RTP withdraw tests
<code>BankDropdown</code>	Patron configured for bank dropdown management tests
<code>NetspendTransactions</code>	Patron with Netspend account for funding source tests
<code>FourBanks</code>	Patron at the 4-bank maximum
<code>operator3</code>	Patron specific to OPERATOR3 configuration
<code>returnURL</code>	Patron configured with a return URL for redirect tests

SessionType	Notes
<code>lowBalance / lowBalanceQA</code>	Patron with low bank balance for limit tests
<code>InvalidAccountInquiry</code>	Patron that triggers account inquiry error
<code>defaultAccount</code>	Patron for default account selection tests

*This document was generated from live code review of both automation frameworks.
For questions, contact the QA team or Brock Pivrotto (brock.pivrotto@pavilionpay.com).*